BÁO CÁO 02

So sánh OpenCode, Pi và Codex CLI

Báo cáo so sánh OpenCode, Pi và Codex CLI theo kiến trúc và cơ chế hoạt động, khảo sát sự thay đổi giữa OpenCode 1.x và 2.x, đồng thời đánh giá ba công cụ trên cùng một bộ nhiệm vụ. Các nhận định kỹ thuật có liên kết đến mã nguồn cụ thể; kết quả thực nghiệm đi kèm script, dữ liệu đầu vào, log và bản sửa để đối chiếu.

Phân tích mã nguồn, so sánh phiên bản và 27 lượt thực nghiệm

Nội dung

1. So sánh OpenCode, Pi và Codex CLI 3

1.1. Kiến trúc . . . 3

1.2. Vòng lặp agent . . . 3

1.3. Hệ thống tool . . . 4

1.4. Permission, phê duyệt và sandbox . . . 4

1.5. Quản lý context . . . 5

1.6. Khả năng mở rộng . . . 6

2. Thực nghiệm OpenCode 1.19.5 và 2.2.7 6

2.1. Chọn phiên bản và dựng môi trường . . . 6

2.2. Kiến trúc và background service chung . . . 7

2.3. API/SDK mới và script đã chạy . . . 8

2.4. Permission - thay đổi dạng cấu hình và độ ưu tiên . . . 10

2.5. Subagent và shell chạy nền . . . 10

2.6. Port plugin V1 sang V2 . . . 12

2.7. Tổng hợp bằng chứng và giới hạn thực nghiệm . . . 13

2.8. Tái lập các lượt chạy . . . 13

3. Thực nghiệm cùng bộ task trên OpenCode, Pi và Codex CLI 14

3.1. Bộ dữ liệu và yêu cầu của từng task . . . 14

3.2. Cấu hình và quy trình chạy . . . 15

3.3. Cách đo token và thời gian . . . 17

3.4. Kết quả tổng hợp . . . 17

3.5. Dữ liệu từng lượt và đánh giá kết quả . . . 19

3.6. Phạm vi diễn giải kết quả . . . 21

3.7. Mã chạy đầy đủ và cách tái lập . . . 22

1. So sánh OpenCode, Pi và Codex CLI

Phạm vi phần này là **OpenCode V2**, **Pi Coding Agent** và **Codex CLI**, đọc theo ba snapshot dưới đây. Các kết luận mô tả đường thực thi trong source tại các commit này. OpenCode V1 được khảo sát riêng ở mục 2.

Công cụ	Snapshot mã nguồn
OpenCode V2	`12a661e`
Pi	`ea1cfcc`
Codex CLI	`8703d32`

1.1. Kiến trúc

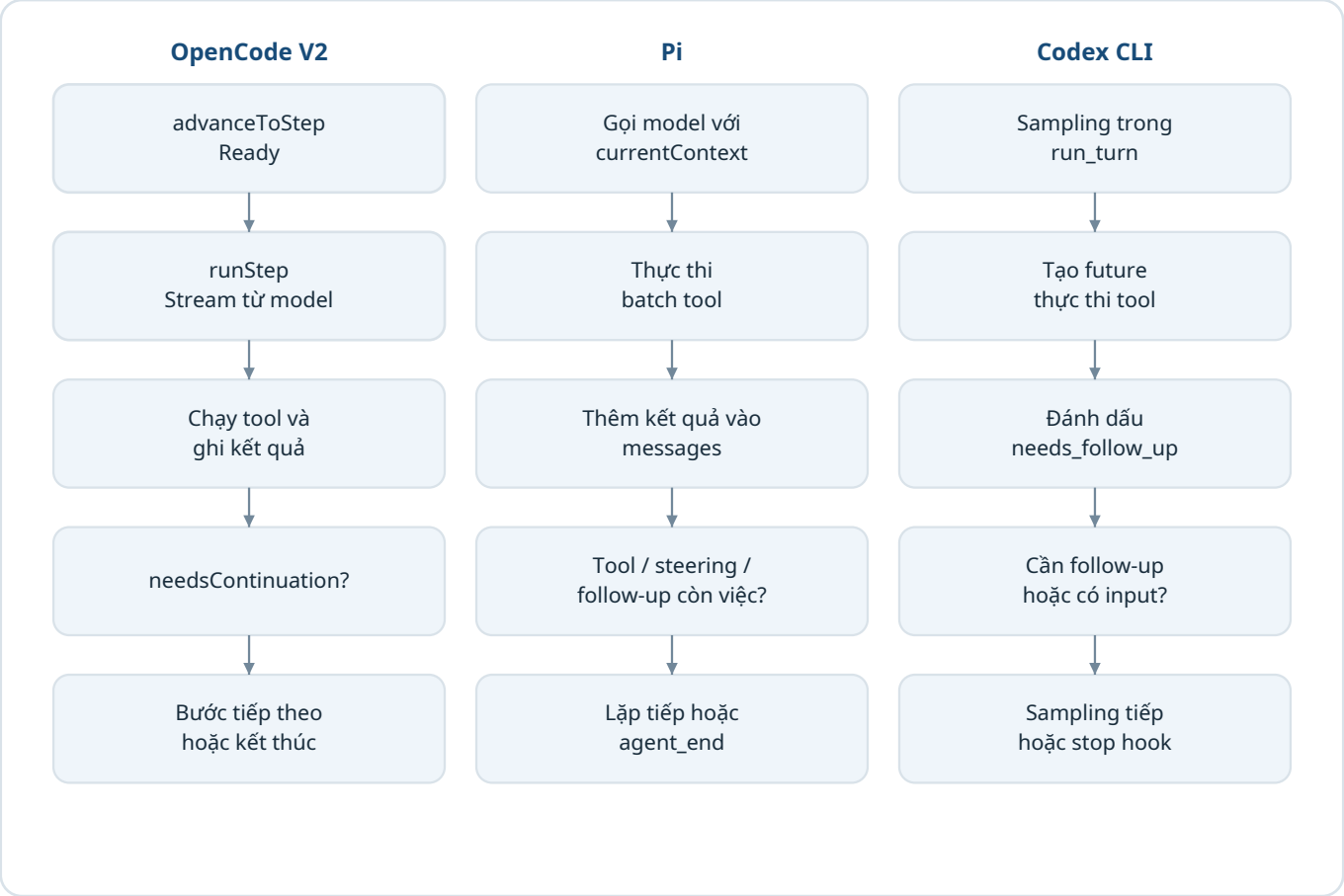
- **OpenCode** có ranh giới HTTP-core rõ ở đường gửi prompt. Handler `session.prompt` lấy session ID và payload từ request, chuyển chúng vào dịch vụ `session.prompt` rồi ánh xạ lỗi miền sang lỗi HTTP. Trong core, `SessionContext.select` đọc session, kiểm tra location, chọn agent và lấy tập tool cùng chỉ dẫn. Runner dùng context đó để chuẩn bị từng bước. HTTP handler · Chọn context · Chuẩn bị bước
- **Pi** tổ chức ứng dụng quanh session runtime dùng chung. Entry point tạo session từ các dịch vụ, tạo runtime và chuyển runtime vào chế độ RPC hoặc print. Lõi vòng lặp nằm ở package `agent`, còn lớp `AgentSession` của coding-agent xử lý việc lưu message. Hai lớp này lần lượt đảm nhiệm việc chạy model, công cụ và quản lý phiên của ứng dụng. Khởi tạo runtime · RPC và print · Agent loop · Lưu message
- **Codex CLI** có entry point Rust chuyển nhánh `exec` sang `codex_exec::run_main` và có đường khởi chạy `codex_tui::run_main` cho TUI. Ở core, tác vụ regular gọi `run_turn` với session, turn context, input và cancellation token. Khi khảo sát hành vi của agent, `run_turn` là điểm đọc phù hợp hơn phần phân tích tham số của CLI. CLI exec · CLI TUI · Regular task · Turn

1.2. Vòng lặp agent

Cùng xét tình huống model yêu cầu đọc file rồi cần thêm một lượt để trả lời.

Công cụ	Cách vòng lặp xử lý tình huống
OpenCode	`advanceToStep` chuẩn bị bước kế tiếp. Khi trả về `Ready`, runner chạy `runStep`. Bên trong bước, stream từ model phát sự kiện `tool-call`; nếu provider chưa thực thi, core chạy tool, giới hạn output và chuyển kết quả hoặc lỗi cho publisher. Runner dùng `needsContinuation` của bước để điều phối tiếp. Vòng lặp · Xử lý stream · Quyết định tiếp tục
Pi	Lõi agent thực thi batch tool call, đưa từng tool result vào `currentContext.messages` và tiếp tục nếu batch chưa yêu cầu kết thúc. Sau mỗi lượt, `finishTurn` và các hàng đợi steering, follow-up có thể thay đổi bước tiếp theo. Nếu model dừng vì giới hạn độ dài, Pi tạo lỗi cho các tool call trong message đó thay vì thực thi batch có thể chứa tham số bị cắt. Xử lý batch và điều kiện dừng
Codex CLI	Khi một response item được nhận diện là tool call, core tạo future thực thi và bật `needs_follow_up`. Sau sampling, `run_turn` kết hợp cờ này với việc còn input đang chờ. Khi cả hai đều hết, code chuyển sang xử lý stop hook và kết thúc turn theo kết quả của nhánh đó. Tạo tool future · Quyết định follow-up · Nhánh stop

Điểm khác biệt trong cách biểu diễn trạng thái thể hiện ngay ở ba đường đọc trên. OpenCode truyền trạng thái tiếp tục giữa các step. Pi thêm tool result trực tiếp vào message array của vòng lặp. Codex gộp nhu cầu follow-up do model với input đang chờ trong turn. OpenCode · Pi · Codex



Hình 1. Ba đường xử lý khi model gọi tool. Sơ đồ rút gọn các nhánh lỗi và hủy. Điều kiện tiếp tục ở Pi còn xét steering, follow-up và quyết định từ `finishTurn`. OpenCode · Pi · Codex CLI

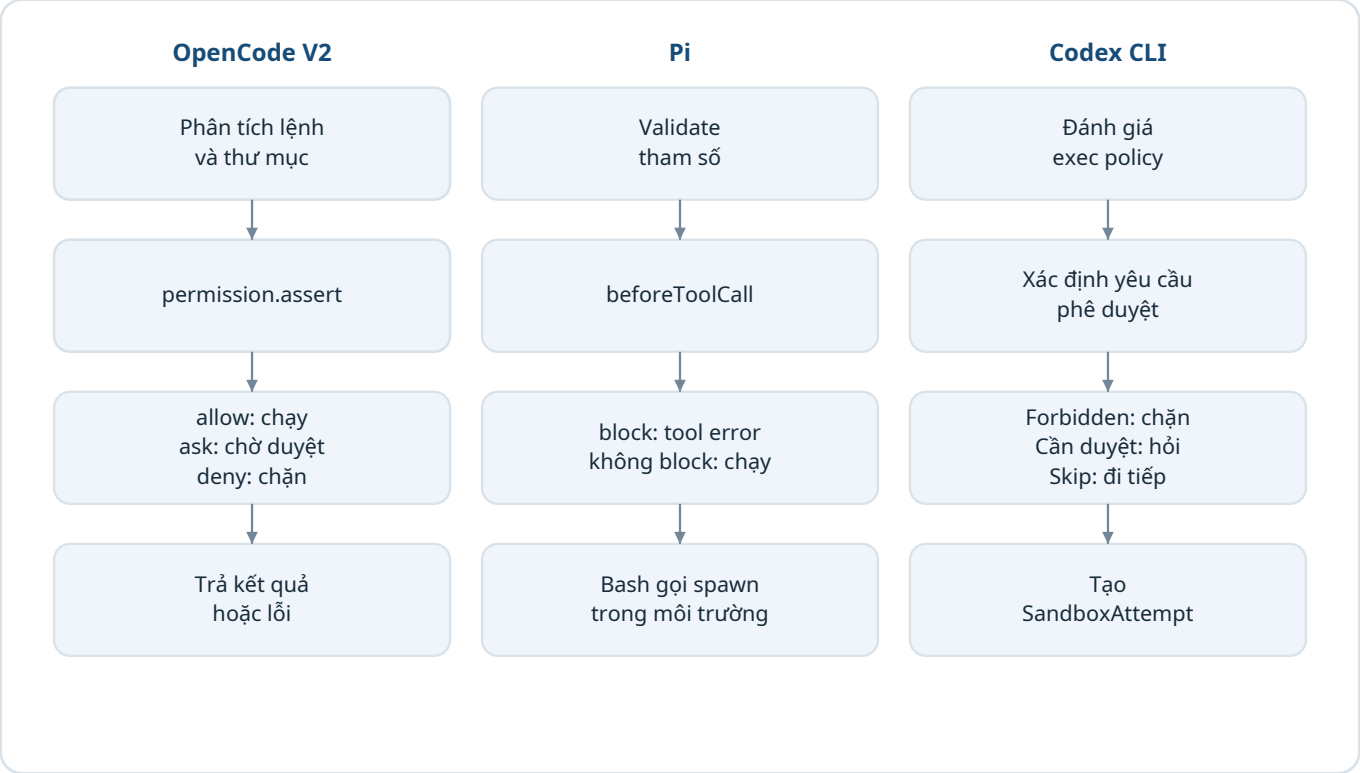
1.3. Hệ thống tool

Công cụ	Từ định nghĩa đến thực thi
OpenCode	Dịch vụ tool chuyển schema khi đăng ký và cung cấp snapshot cho session. Tool `read` là ví dụ về việc khai báo thao tác và triển khai `execute` cùng kiểm tra quyền ngay tại tool. Bước chạy model chuyển tool call cục bộ đến executor rồi giới hạn kích thước đầu ra. Đăng ký tool · Snapshot của session · Read tool · Thực thi
Pi	`AgentSession` gộp built-in tool với tool từ extension, sau đó chọn tool hoạt động. Ví dụ `bash` khai báo tham số bằng TypeBox. Lõi agent tìm tool theo tên, chuẩn bị và validate tham số trước khi chạy hook. Việc chạy một batch tuần tự hay song song có nhánh riêng trong agent loop. Registry và active tools · Bash schema · Validate · Chọn cách chạy batch
Codex CLI	`ToolRouter` giữ registry và cung cấp danh sách spec model nhìn thấy. Router chuyển response item thành tool call, rồi dispatch qua registry. Cách tách này cho phép theo dõi riêng spec gửi cho model và đường xử lý khi model thực sự gọi tool. Spec model nhìn thấy · Dựng tool call · Dispatch

1.4. Permission, phê duyệt và sandbox

Phân tích quyền thực thi cần xét ba lớp kiểm soát: công cụ được cung cấp cho model, yêu cầu phê duyệt đối với từng thao tác và giới hạn của tiến trình chạy lệnh.

- **OpenCode** hợp nhất quyền agent và session khi dựng snapshot tool. Khi shell thực thi, nó phân tích lệnh và đường dẫn, kiểm tra truy cập thư mục ngoài phạm vi và gọi `permission.assert` với các tài nguyên lệnh đã phân tích. Hàm `evaluate` tìm rule khớp cuối cùng. Nhánh `assert` trả lỗi khi `deny`, đi tiếp khi `allow`, hoặc tạo yêu cầu rồi chờ quyết định khi `ask`. Snapshot · Shell authorization · So khớp rule · Xử lý quyết định
- **Pi** kiểm soát tool khả dụng qua registry và tập active tool. Trên đường thực thi model gọi tool, `beforeToolCall` có thể trả `block`; lõi chuyển quyết định thành tool error. Lớp extension chuyển sự kiện `tool_call` tới handler và trả ngay khi một handler chặn. Ở triển khai bash mặc định được khảo sát, Pi gọi `spawn` với shell, cwd và môi trường. Hook chặn trước khi chạy và quyền của tiến trình con là hai điểm kiểm soát riêng. Active tools · Chuyển sự kiện từ session · Hook chặn · Trả tool error · Spawn shell
- **Codex CLI** đánh giá policy lệnh thành yêu cầu phê duyệt, chẳng hạn `Forbidden` hoặc `Skip`; quyết định `Prompt` được xử lý cùng cấu hình approval. Tool orchestrator kiểm tra yêu cầu này, gọi bước phê duyệt khi cần và xây dựng `SandboxAttempt` cho lần thực thi. Khi đọc một trường hợp lệnh bị chặn, cần xác định nó dừng ở policy, phê duyệt hay sandbox. Ánh xạ quyết định policy · Xử lý approval · Tạo sandbox attempt



Hình 2. Các điểm kiểm soát trên đường chạy một lệnh shell. Mỗi cột mô tả luồng riêng của một công cụ. Các nhánh từ chối sau khi hỏi được lược bớt. OpenCode shell · Pi tool hook · Codex approval

1.5. Quản lý context

Công cụ	Context gửi model và compaction
OpenCode	Mỗi step ước lượng kích thước prompt, gọi compaction và chuẩn bị transcript từ chỉ dẫn, tool và message đã load. Nếu compaction hoàn tất, runner quay lại chuẩn bị bước. Khi chạm giới hạn bước của agent, nó thêm chỉ dẫn kết thúc và đặt `toolChoice` thành none. Chuẩn bị model request
Pi	Lõi agent cho phép `transformContext` xử lý message trước khi `convertToLlm` chuyển sang dạng gửi provider. Ở lớp session, Pi kiểm tra ngưỡng token và có đường xử lý compaction do threshold hoặc overflow. Khi lưu kết quả nén, session ghi cả summary và ID entry đầu tiên cần giữ, rồi làm mới projection. Chuyển đổi context · Ngưỡng · Lưu compaction
Codex CLI	`run_turn` có nhánh kiểm tra compaction trước sampling. Sau sampling, core thu trạng thái token và input đang chờ. Nếu chọn rollover giữa turn, core gọi `run_auto_compact` và có thể tiếp tục turn sau khi nén. Đây là điểm cần xem khi phân tích vì sao cùng một tác vụ dài có thêm lượt gọi model. Pre-sampling · Trạng thái sau sampling · Mid-turn compaction

1.6. Khả năng mở rộng

- **OpenCode** plugin host cung cấp bề mặt `tool.transform`, `tool.reload`, `tool.list` và hook tool. Host cũng có bề mặt thao tác với cấu hình MCP. Hai điểm mở rộng có vai trò khác nhau: plugin có thể tác động vào registry và lifecycle, còn MCP đưa tích hợp bên ngoài vào hệ thống. Tool host · MCP host
- **Pi** extension gọi `registerTool` để đăng ký tool và có thể lắng nghe `tool_call`. `AgentSession` hợp nhất các tool đã đăng ký với built-in tool. Đây là đường thêm hành vi qua extension mà không cần sửa lõi `agent-loop`. Đăng ký tool · Ghép registry · Tool-call hook
- **Codex CLI** có hai điểm mở rộng liên quan trực tiếp đến đường tool đã khảo sát. Khâu lập tool plan thêm MCP tool vào registry. Khâu thực thi chạy hook trước và sau tool theo hợp đồng riêng. MCP cung cấp thao tác để model gọi, còn hook can thiệp vào vòng đời của lời gọi. Ghép MCP tool vào plan · PreToolUse · PostToolUse

2. Thực nghiệm OpenCode 1.19.5 và 2.2.7

Phần này khảo sát năm thay đổi qua mã nguồn đúng phiên bản và các lượt chạy trên Linux x64 ngày **11/11/2026**. Hai bản được cài thành hai binary độc lập, chạy server đồng thời trên hai cổng loopback và dùng chung model miễn phí `opencode/minimax-m3.1-flash-free`. Script, cấu hình, plugin và log nằm trong thư mục thực nghiệm.

2.1. Chọn phiên bản và dựng môi trường

Metadata npm của `opencode-ai` và `@opencode/cli` được lọc theo major, loại prerelease rồi lấy phiên bản semver lớn nhất. Kết quả ở thời điểm tải là `1.19.5` và `2.2.7`. Việc chọn theo major cần thiết vì hai nhánh dùng package phát hành khác nhau. Mốc "cuối" và "mới nhất" trong báo cáo được hiểu tại thời điểm thực nghiệm. Bản ghi cài đặt lưu phiên bản, commit, package nền tảng và SHA-256 của archive lẫn binary. Manifest cài đặt

Thành phần	OpenCode V1	OpenCode V2
Phiên bản	`1.19.5`	`2.2.7`
Package CLI	`opencode-ai`	`@opencode/cli`
Package binary Linux x64	`opencode-linux-x64@1.19.5`	`@opencode/cli-linux-x64@2.2.7`
Commit của tag	`810c0eba1a7348036f8bdf61e6632d0c36c6876c`	`fedc6f208aa2bd54c1019227214568083f6380ad`
Binary trong lab	`/tmp/opencode-lab/v1/package/bin/opencode`	`/tmp/opencode-lab/v2/package/bin/opencode`
Server thử API	`127.0.0.1:14195`	`127.0.0.1:14227`
Repo thử	`/tmp/opencode-lab/project-v1`	`/tmp/opencode-lab/project-v2`
Thư mục dữ liệu và cấu hình	`/tmp/opencode-lab/home-v1`	`/tmp/opencode-lab/home-v2`

Git đã có sẵn. Binary Linux được tải trực tiếp từ npm và giải nén vào thư mục lab. Bun `1.4.4` được dùng để cài dependency và chạy TypeScript SDK. Model được lấy từ catalog mà server V2 trả về và được gọi thành công trên cả hai bản. Không cần nhập API key cá nhân cho các lượt chạy này. Catalog model thu được · Kết quả V1 · Kết quả V2

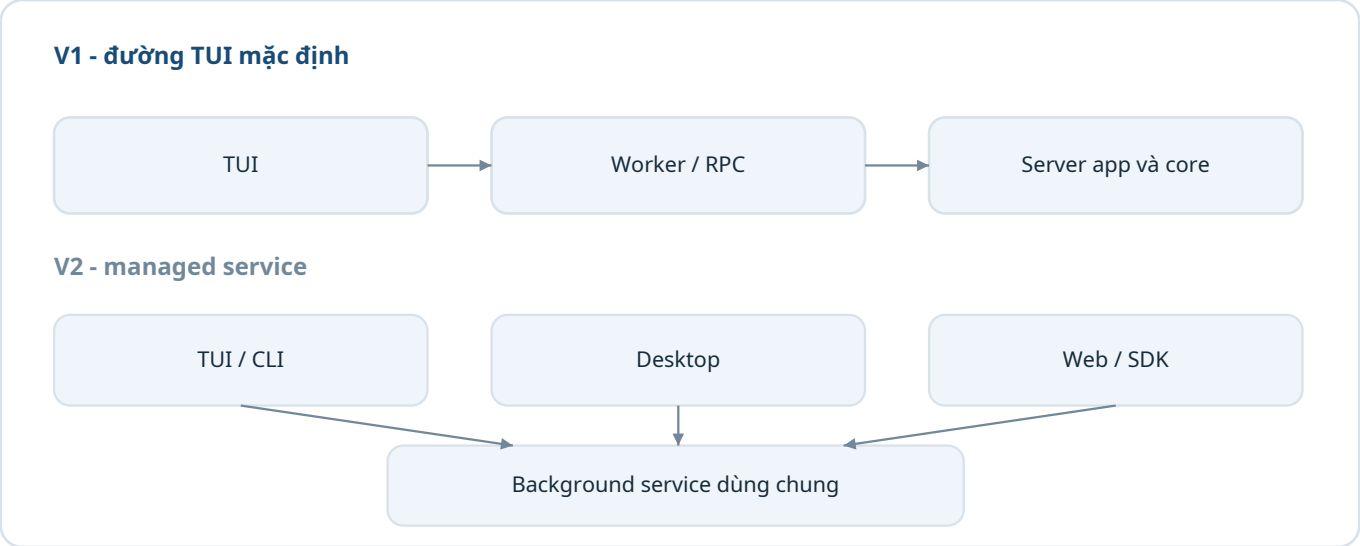
Các biến `XDG_CONFIG_HOME`, `XDG_DATA_HOME`, `XDG_CACHE_HOME`, `XDG_STATE_HOME` và `OPENCODE_TEST_HOME` tách đường dẫn cho từng tiến trình. `OPENCODE_TEST_HOME` có điểm hỗ trợ trong source của cả hai bản. Các binary được gọi bằng đường dẫn đầy đủ. Nhờ đó, chạy V1 và V2 không phụ thuộc vào bản nào đang đứng trước trong `PATH`. Global path V1 · Global path V2 · Mã thiết lập môi trường lab

Trong lần khởi tạo đầu, V1 có lỗi HTTP 500 và log cũ ghi nhận lỗi tải catalog/certificate. Sau khi dùng dependency đã cài cục bộ, tách home và khởi động lại server, các ca V1 chạy thành công. Chưa tách riêng được một nguyên nhân duy nhất cho lỗi khởi tạo đó. Kết quả ở các mục sau lấy từ lượt chạy thành công, còn log server được giữ để kiểm tra quá trình xử lý lỗi. Dependency môi trường plugin V1 được tái sử dụng bằng symlink và lock metadata từ các package đã cài trong lab. Setup tái lập · Log server V1 · Log bootstrap V1

2.2. Kiến trúc và background service chung

Ở V1, đường chạy TUI mặc định tạo một `Worker` rồi dùng RPC để giao tiếp với worker đó. Worker cung cấp thao tác `fetch` bằng cách chuyển request đến ứng dụng server trong tiến trình, đồng thời có thể mở HTTP listener khi được yêu cầu. Vì vậy, V1 đã có server API và cơ chế client-server. Đặc điểm cần đối chiếu khi chuyển V2 là cách tổ chức và khám phá service dùng chung. TUI tạo worker V1 · Worker xử lý request và listener

Ở V2, resolver của CLI chọn giữa endpoint chỉ định qua `--server`, server riêng qua `--standalone`, và managed service. Với đường managed mặc định, nó gọi `Service.ensure`. Hàm này kiểm tra registration, phiên bản và sức khỏe service, rồi tái sử dụng hoặc khởi chạy service theo kết quả. Desktop cũng gọi `Service.ensure` và dùng lệnh CLI `serve --service` khi cần tạo tiến trình server. CLI chọn kết nối · Cơ chế ensure · Desktop kết nối service



Hình 3. Sơ đồ tổng hợp từ các đường kết nối trong source. SDK thử nghiệm kết nối vào endpoint của managed service; đường kết nối Desktop được xác minh bằng mã nguồn. V1 worker · V2 CLI · V2 Desktop

Bài thử managed service dùng XDG riêng dưới `/tmp/opencode-lab/managed` và cổng `14333`. Script gọi `Service.ensure` lần thứ nhất, tạo session bằng client A, gọi `ensure` lần thứ hai rồi đọc lại session bằng client B. Hai lần đọc `server.info` trả cùng PID. Session ID của A và B cũng trùng nhau. HTTP GET trang web trả `200` và `Content-Type: text/html`. Script dừng service sau khi thu dữ liệu. Script service · Bằng chứng service

TUI thật cũng được mở qua pseudo-terminal với endpoint và session ID của service này. Bản ghi terminal có tiền tố tiêu đề `Shared lab sessio` do tab giới hạn chiều rộng, không có lỗi kết nối và tiến trình thoát với mã 0 sau khi nhận thao tác đóng. Desktop chưa được khởi chạy trong lab; nhận định về Desktop dựa trên đường `Service.ensure` trong source. Bản ghi terminal · Kết quả TUI và service

Việc dùng một service chung cũng có nghĩa là phiên và tác vụ thuộc về server mà client kết nối tới. Khi cần tái lập thử nghiệm độc lập, có thể chọn `--standalone` hoặc endpoint riêng. Trong lab này, ca so sánh tool dùng hai HTTP server riêng, còn ca kiến trúc dùng managed service để kiểm tra đúng cơ chế tái sử dụng tiến trình. Nhánh standalone và managed · Driver hai server

2.3. API/SDK mới và script đã chạy

Hai bản dùng các hợp đồng request khác nhau trên đường gửi prompt. V1 có các endpoint tạo session và gửi message dưới `/session`; SDK tương ứng là `@opencode-ai/sdk`. V2 dùng `@opencode/client`, có endpoint `/api/session` và `/api/session/:sessionID/prompt`. Đọc đúng package và endpoint giúp tránh nhầm SDK khi chuyển phiên bản. Endpoint V1 · Package SDK V1 · Client tạo session V2 · Client gửi prompt V2

Thao tác	V1	V2
Tạo client	`createOpencodeClient(...)`	`OpenCode.make(...)`
Tạo session	SDK nhận options theo `body`, `query`	`session.create({ location, model, ... })`
Gửi prompt trong lab	`POST /session/:id/message` với `parts`	`session.prompt({ sessionID, text })`
Nhận event trong lab	SSE `/event`, dữ liệu trong `properties`	`event.subscribe()`, dữ liệu trong `data`
Theo dõi hoàn thành	Đọc response message và lịch sử phiên	Theo dõi event kết thúc execution của đúng session

Bảng đối chiếu dựa trên request, response và sự kiện thu được từ môi trường thử nghiệm. Ở V2, `session.prompt` trả về một mục inbox đã tiếp nhận. Bài thử tiếp tục chờ event kết thúc execution để lấy phản hồi của model. Điều này quan

trọng với script tự động hóa vì nhận được xác nhận gửi prompt chưa chứng minh model đã trả lời xong. Request schema V2 · Bản ghi API V2 · Event V1

Script dùng Promise SDK V2, mở subscription trước khi gửi prompt và chờ `server.connected` để tránh bỏ lỡ những event đầu tiên. Sau đó tạo session, chọn model, gửi prompt, lọc event theo `data.sessionID` và lưu log JSONL. Source server cho thấy subscription dùng SSE, có frame `server.connected` và heartbeat. SSE handler · SDK event subscription

```
import { OpenCode } from '@opencode/client';
import { appendFileSync } from 'node:fs';
const output = new URL('../evidence/api-v2-events.jsonl', import.meta.url);
await Bun.write(output, '');
const client = OpenCode.make({
  baseUrl: process.env.LAB_SERVER ?? 'http://127.0.0.1:14227',
  headers: { Authorization: 'Basic ' + btoa('opencode:lab-local-only') },
});
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(new Error('Lab timeout after 120 seconds')), 120_000);
let sessionID: string | undefined;
let connected!: () => void;
const ready = new Promise<void>(resolve => { connected = resolve; });
const events: unknown[] = [];
const listening = (async () => {
  for await (const event of client.event.subscribe({ signal: controller.signal })) {
    if (event.type === 'server.connected') connected();
    if (!sessionID || (event as any).data?.sessionID !== sessionID) continue;
    events.push(event);
    appendFileSync(output, JSON.stringify(event) + '\n');
    if (['session.execution.succeeded', 'session.execution.failed',
'session.execution.interrupted'].includes(event.type)) return event;
  }
})();
try {
  await Promise.race([ready,listening.then(()=>{throw new Error("Event stream closed before ready");})]);
  const session = await client.session.create({
    title: 'Migration lab API smoke',
    location: { directory: '/tmp/opencode-lab/project-v2' },
    model: { providerID: 'opencode', id: process.env.LAB_MODEL ?? 'minimax-m3.1-flash-free' },
  });
  sessionID = session.id;
  const start = performance.now();
  const admitted = await client.session.prompt({ sessionID, text: 'Reply exactly MIGRATION_API_OK. Do not use
tools.' });
  const terminal = await listening;
  if (!terminal) throw new Error("Event stream ended without an execution result");
  const context = await client.session.context({ sessionID });
  const summary = { version: '2.2.7', sessionID, admitted, terminal, elapsedMs:
Math.round(performance.now()-start), context, eventCount: events.length };
  await Bun.write(new URL('../evidence/api-v2-summary.json', import.meta.url), JSON.stringify(summary, null,
2));
  console.log(JSON.stringify({sessionID,eventCount:events.length,terminal,elapsedMs:summary.elapsedMs}));
} finally {
  clearTimeout(timer);
  controller.abort();
  await listening.catch(() => undefined);
}
```

Đoạn mã trên là script đã dùng trong lab. Toàn bộ file và dữ liệu đầu ra được lưu cùng báo cáo. sdk-smoke.ts · Event JSONL

Kết quả lượt smoke test nhận được `MIGRATION_API_OK`, gồm **13 event của session** và kết thúc bằng `session.execution.succeeded`. Thời gian từ lúc gửi prompt đến khi nhận kết quả là **2.637 ms**. Usage ghi `input=3647`, `output=7`, `reasoning=0`, `cache.read=1792`, `cache.write=0`, cost báo về bằng 0. Các trường được giữ theo cách provider trả về. Kết quả và context

2.4. Permission - thay đổi dạng cấu hình và độ ưu tiên

V1 cho phép gom quy tắc theo tool trong object. Hàm `fromConfig` chuyển object thành danh sách rule bằng thứ tự `Object.entries`. Khi đánh giá, V1 cũng dùng `findLast`. V2 biểu diễn native bằng mảng rule có ba trường `action`, `resource`, `effect`, và hàm `evaluate` tiếp tục tìm rule khớp cuối cùng. Vì vậy, chuyển V2 làm thứ tự rule hiển thị trực tiếp trong cấu hình; thứ tự cũng đã có ý nghĩa trong triển khai V1 được chọn. Chuyển config V1 · Evaluate V1 · Evaluate V2

Cấu hình V1 của bài thử dùng tên tool `bash`.

```
{
  "permission": {
    "bash": {
      "*": "allow",
      "printf LAB_ASK": "ask",
      "printf LAB_DENY": "deny"
    }
  }
}
```

Cấu hình V2 tương ứng dùng action `shell`.

```
{
  "permissions": [
    { "action": "shell", "resource": "*", "effect": "allow" },
    { "action": "shell", "resource": "printf LAB_ASK", "effect": "ask" },
    { "action": "shell", "resource": "printf LAB_DENY", "effect": "deny" }
  ]
}
```

Các cấu hình đầy đủ còn chứa model và đường dẫn plugin. V2 có bước normalize nhận cả cấu hình quyền cũ và mới, rồi ghép rule của `tools`, `permission` và `permissions` theo thứ tự. Do đó, khả năng chấp nhận cấu hình cũ và khả năng chạy plugin cũ cần được kiểm tra riêng. Config V1 · Config V2 · Normalize V2

Ca chạy	V1 1.19.5	V2 2.2.7	Căn cứ nghiệm thu
`printf LAB_ALLOW`	Chạy, trả `LAB_ALLOW`	Chạy, trả `LAB_ALLOW`	Tool hoàn thành và output khớp
`printf LAB_ASK`	Có một yêu cầu duyệt	Có một yêu cầu duyệt	Có `permission.asked`, script trả `once`, sau đó tool hoàn thành
`printf LAB_DENY`	Tool trả lỗi rule cấm	Tool trả `permission.rejected`	Không có tool success của lệnh bị cấm

Script tự động duyệt **đúng lệnh** `printf LAB_ASK` **một lần** để thực nghiệm chạy xuyên suốt. Các yêu cầu khác được từ chối. Ở ca `deny`, execution của V2 vẫn có thể kết thúc thành công sau khi model thông báo lệnh bị chặn. Vì vậy, trạng thái thành công của execution cần được đọc cùng kết quả tool để xác định lệnh có thực thi hay không. Trace ask V1 · Trace ask V2 · Trace deny V2

Bài thử thứ tự V2 dùng cùng lệnh `printf LAB_ORDER` trong hai session. Khi rule rộng `shell/*/ask` đứng trước và rule cụ thể `allow` đứng sau, không phát sinh yêu cầu duyệt. Khi đảo vị trí, có một `permission.asked`. Hai lượt xác nhận bằng runtime rằng rule khớp cuối cùng quyết định kết quả trong trường hợp này. Script đảo rule · Log hai thứ tự

2.5. Subagent và shell chạy nền

V1 `1.19.5` đã triển khai subagent nền, nhưng kiểm tra cờ `OPENCODE_EXPERIMENTAL_BACKGROUND_SUBAGENTS`. Khi cờ tắt, schema gửi model chỉ chứa các tham số cơ bản và phần mô tả không bổ sung hướng dẫn chạy nền. Khi bật, `background=true` có thể đi vào nhánh chạy nền. Điều kiện experimental · Schema theo cờ · Nhánh trả nền

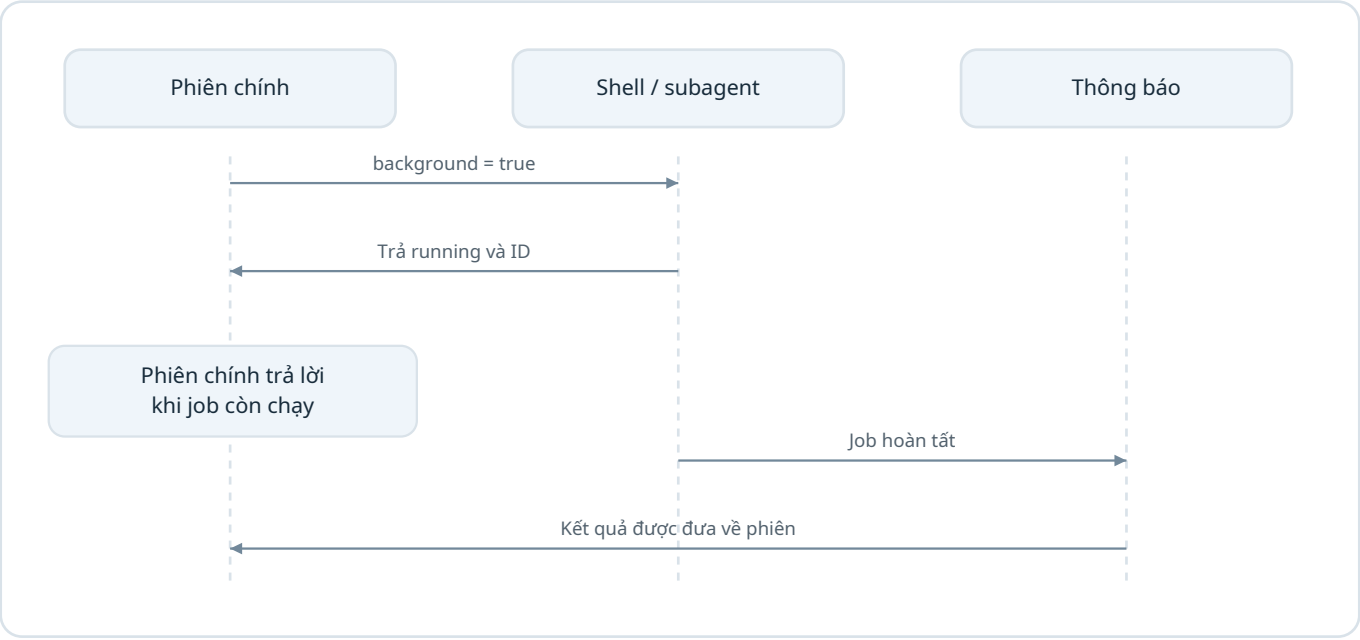
Trong lượt V1 mặc định, model gọi `task` mà không truyền `background`; tool đợi child hoàn thành rồi mới trả về. Lượt riêng có bật cờ đã gọi `task` với `background=true`, trả `<task ... state="running">` trong **13 ms**, sau đó phiên chính trả

PARENT_FREE. Lịch sử phiên có thông báo child hoàn thành cùng marker CHILD_DONE. Hai lượt này phân biệt được hành vi mặc định với khả năng experimental đã tồn tại ở V1. V1 mặc định · V1 bật experimental

V2 đưa tham số background vào input schema của cả subagent và shell tại các điểm đọc được khảo sát. Subagent có nhánh khởi chạy công việc nền và trả session ID. Shell có nhánh chuyển job sang nền và trả shell ID cùng file output. Shell nền mặc định không đặt timeout nếu người gọi không cung cấp timeout riêng. Subagent input và mô tả · Subagent thực thi · Shell input · Shell timeout · Shell trả nền

Ca V2	Thao tác	Tool trả sau	Phiên chính kết thúc lượt đầu sau	Kết thúc lượt xử lý thông báo sau
Shell nền	sleep 5; printf LAB_BACKGROUND_DONE với background=true	9 ms	1.379 ms	8.877 ms
Subagent nền	general, background=true; child chạy sleep 5; printf CHILD_DONE	7 ms	1.757 ms	10.404 ms

Các mốc V2 tính từ session.tool.called. "Kết thúc lượt xử lý thông báo" bao gồm thời gian agent nhận thông báo và gọi model để phản hồi. Nó có thể muộn hơn thời điểm tiến trình con thực sự thoát. Ở shell nền, phiên chính đã trả MAIN_FREE trước khi lệnh sleep 5 có thể hoàn tất. Ở subagent nền, nó trả PARENT_FREE trước khi child báo kết quả. Mốc thời gian trích từ log · Trace shell · Trace subagent



Hình 4. Luồng được quan sát trong các trace V2. Subagent V1 khi bật experimental cũng trả trạng thái chạy rồi gửi thông báo hoàn thành về sau. Bằng chứng V2 · Bằng chứng V1

Schema shell V1 ở bản đã chọn khai báo command, timeout, workdir; không có tham số native background như tool shell V2. Đây là khác biệt trên giao diện tool được khảo sát. Báo cáo chưa thử các cách quản lý tiến trình khác của V1 để kết luận toàn bộ khả năng chạy nền của hệ điều hành hay các API experimental liên quan. Schema shell V1 · Schema shell V2

2.6. Port plugin V1 sang V2

Plugin thử nghiệm thêm tool `lab_echo`. Với input `PORT_OK`, tiêu chí nghiệm thu là tool thực sự chạy và trả `MIGRATION_PLUGIN:PORT_OK`. Chuỗi marker cho phép kiểm tra từ tool output trong log, thay vì chỉ dựa vào lời model mô tả.

Plugin V1 export một hàm khởi tạo, trả object chứa hook và định nghĩa tool. Helper `tool` nhận `args` theo Zod cùng hàm `execute`; kết quả có thể là chuỗi. Kiểu plugin V1 · Helper tool V1

```
import { tool } from '@opencode-ai/plugin';

export default async () => ({
  tool: {
    lab_echo: tool({
      description: 'Return the migration lab marker followed by the supplied text.',
      args: { text: tool.schema.string() },
      async execute({ text }) {
        return `MIGRATION_PLUGIN:${text}`;
      },
    }),
  },
});
```

Plugin V2 dùng default export là một định nghĩa có `id` cùng `setup` hoặc `effect`. Với Promise API, `setup` đăng ký tool qua `ctx.tool.transform`. Tool mới khai báo schema input/output bằng Effect Schema và trả giá trị output cùng content đưa về model. Loader V2 kiểm tra hình dạng module trước khi khởi tạo plugin. Định nghĩa Promise plugin · Ví dụ tool trong SDK plugin · Schema module

```
import { Plugin } from '@opencode/plugin';
import { Schema } from 'effect';

export default Plugin.define({
  id: 'lab.echo',
  async setup(ctx) {
    await ctx.tool.transform(editor => {
      editor.add({
        name: 'lab_echo',
        description: 'Return the migration lab marker followed by the supplied text.',
        options: { codemode: false },
        input: Schema.Struct({ text: Schema.String }),
        output: Schema.Struct({ text: Schema.String }),
        async execute({ text }) {
          const result = `MIGRATION_PLUGIN:${text}`;
          return { output: { text: result }, content: result };
        },
      });
    });
  },
});
```

Các thay đổi khi port gồm package import, default export, vị trí đăng ký tool, schema tham số và cấu trúc kết quả. `options.codemode=false` được đặt để tool thử nghiệm xuất hiện trực tiếp trong danh sách model gọi. Plugin V2 được cấu hình bằng đường dẫn thư mục chứa `index.ts`. Plugin V1 hoàn chỉnh · Plugin V2 hoàn chỉnh · Cách resolve entrypoint local

Trường hợp	Kết quả thực tế
Plugin V1 trên OpenCode 1.19.5	`lab_echo` hoàn thành và trả `MIGRATION_PLUGIN:PORT_OK`
Plugin đã port trên OpenCode 2.2.7	Plugin `lab.echo` ở trạng thái `active`; tool trả cùng marker
Plugin V1 nguyên bản trên OpenCode 2.2.7	Catalog plugin ghi `failed`; loader báo thiếu định nghĩa đúng hợp đồng

Ca không tương thích giữ nguyên code V1, đặt trong một thư mục local hợp lệ cho V2, reload location rồi khởi tạo session để buộc runtime nạp plugin. Lỗi thu được là `Plugin must export a default definition with an id and an effect or setup function`. Source loader tạo đúng lỗi này khi module không khớp schema. Sau ca thử, script khôi phục cấu hình về plugin đã port. Script đối chứng · Catalog, event và context thu được · Điểm phát lỗi trong loader

2.7. Tổng hợp bằng chứng và giới hạn thực nghiệm

Hạng mục	Kết quả được xác nhận
Cài song song	Hai binary đúng phiên bản, hai repo thử, hai server riêng, có manifest và checksum
Background service	Hai lần ensure tái sử dụng PID; hai client đọc cùng session; web trả HTML
API/SDK V2	Tạo session, gửi prompt, nhận SSE và phản hồi model thật
Permission	Cả hai bản chạy đủ allow/ask/deny; V2 được thử thêm hai thứ tự rule chồng lấn
Subagent nền	V1 chạy khi bật experimental flag; V2 chạy bằng tham số `background=true`
Shell nền	V2 trả ngay trạng thái running và thông báo hoàn thành về sau
Plugin	V1 gốc chạy trên V1, bản port chạy trên V2, V1 gốc bị loader V2 từ chối

Mỗi ca chức năng chạy một lần thành công sau khi môi trường được thiết lập. Các khoảng thời gian trong mục này chỉ mô tả lượt chạy, chưa đủ để xếp hạng tốc độ hoặc độ ổn định giữa hai phiên bản. Tác vụ chỉ dùng marker và lệnh shell ngắn để tập trung kiểm tra cơ chế. Thử nghiệm hủy tác vụ, recovery sau restart, mất kết nối SSE và tải đồng thời chưa được thực hiện trong phần này. Desktop được đối chiếu từ source; phạm vi runtime được ghi cùng bằng chứng service và TUI. Quan sát tổng hợp từ dữ liệu gốc · Bằng chứng service

2.8. Tái lập các lượt chạy

Chạy từ thư mục `experiments/opencode-v1-v2`. Script setup tải đúng binary Linux x64 đã ghim, kiểm tra checksum, cài dependency theo Bun lockfile và tạo lại cấu hình theo vị trí repo hiện tại.

```
python3 scripts/setup.py

# V2: tạo session, g□i prompt và nhận event
python3 scripts/with_servers.py /tmp/opencode-lab/runtime/bun-linux-x64/bun scripts/sdk-smoke.ts

# V2: plugin, allow/ask/deny, shell n□n và subagent n□n
python3 scripts/with_servers.py /tmp/opencode-lab/runtime/bun-linux-x64/bun scripts/runtime-v2.ts

# V1: plugin, allow/ask/deny và hành vi task mặc định
python3 scripts/with_servers.py /tmp/opencode-lab/runtime/bun-linux-x64/bun scripts/runtime-v1.ts

# V2: đ□o thứ tự rule
python3 scripts/with_servers.py /tmp/opencode-lab/runtime/bun-linux-x64/bun scripts/rule-order.ts

# Plugin V1 trên V2, sau đó V1 subagent có bật cờ experimental
LAB_V1_BACKGROUND=1 python3 scripts/with_servers.py /tmp/opencode-lab/runtime/bun-linux-x64/bun
scripts/final-probes.ts

# Managed service, hai client, trang web và TUI
/tmp/opencode-lab/runtime/bun-linux-x64/bun scripts/shared-service.ts

# T□ng hợp quan sát từ log JSON
python3 scripts/summarize.py
python3 scripts/verify_results.py
```

Các script ghi lại kết quả vào `evidence/`, nên một lượt tái lập sẽ cập nhật các file tương ứng. Model miễn phí trong bài thử phụ thuộc catalog provider tại thời điểm chạy. Nếu model bị gỡ, cần chọn model khả dụng khác và ghi lại thay đổi đó. Các bản cài trong `/tmp/opencode-lab` có thể bị hệ điều hành dọn; script setup dựng lại được binary và môi trường theo manifest. Setup · Bun lockfile

3. Thực nghiệm cùng bộ task trên OpenCode, Pi và Codex CLI

Thực nghiệm dùng một repository Python nhỏ, ba yêu cầu cố định và test nghiệm thu tách khỏi workspace của agent. Mỗi công cụ chạy mỗi task ba lần trên bản sao sạch. Tổng số lượt theo thiết kế là **27**, cùng model `gpt-6.1-sol` và reasoning `medium`. Mỗi lượt được chấm ngay sau khi công cụ dừng, giữ lại log, diff và mã nguồn sau chỉnh sửa để đối chiếu. Dataset và hash đầu vào · Mã điều phối

3.1. Bộ dữ liệu và yêu cầu của từng task

Repository mẫu là **Cartkit**, gồm tính tổng tiền đơn hàng bằng số nguyên cents, báo cáo đơn hàng theo trạng thái và CLI đọc dữ liệu JSON. Dự án dùng Python standard library, có bốn test gốc và dữ liệu đơn hàng mẫu. Cách chọn này giúp lượt chạy tập trung vào đọc code, chỉnh sửa và kiểm tra hành vi. Repository đầu vào · Dữ liệu mẫu · Bốn test gốc

Task	Công việc giao cho agent	Test nghiệm thu	Điểm của code gốc
T1 - sửa lỗi	Sửa cách xử lý quantity bằng 0, giữ mặc định 1 khi thiếu trường và từ chối kiểu dữ liệu không hợp lệ	9	3/9
T2 - thêm tính năng	Thêm parser CSV, gom dòng theo order ID và nối lệnh `import-csv` vào CLI hiện có	12	0/12
T3 - refactor	Gom logic thống kê vào helper dùng chung và hỗ trợ iterable chỉ duyệt được một lần	8	6/8

Tổng cộng có **29 test nghiệm thu** cho ba task. Code gốc vẫn vượt qua cả bốn test sẵn có, trong khi các test bổ sung phát hiện những hành vi cần sửa hoặc chưa được triển khai. T2 đạt 0 vì module và lệnh CSV chưa tồn tại. Kết quả baseline · Log test gốc

T1 - sửa lỗi tính tiền. Biểu thức ban đầu là `line.get('quantity') or 1`. Nó thay giá trị 0 bằng 1, đồng thời che mất các giá trị sai như `None` hoặc `False` trước bước kiểm tra kiểu. Task yêu cầu phân biệt trường bị thiếu với trường có giá trị cụ thể. Test kiểm tra giỏ hàng chỉ có quantity 0, giỏ hỗn hợp, mặc định khi thiếu trường, kiểu không hợp lệ, làm tròn discount half-up, phí vận chuyển và việc giữ nguyên input. Code gốc · Prompt T1 · Test T1

T2 - parser CSV và CLI. Input phải có đúng bốn cột `order_id`, `sku`, `quantity`, `unit_price_cents`; thứ tự header có thể thay đổi. Parser cần trim khoảng trắng, xử lý quoting chuẩn CSV, giữ thứ tự đơn hàng xuất hiện lần đầu và giữ thứ tự các dòng trong từng đơn. Các ca lỗi gồm header thừa/thiếu/trùng, sai số ô, ID rỗng, số âm, số thập phân, chuỗi boolean và chữ số Unicode. Test CLI gọi tiến trình Python rồi parse JSON trả về để kiểm tra tính năng qua entry point người dùng. Prompt T2 · Test T2

Ví dụ input hợp lệ của T2 và output mong đợi được thể hiện dưới đây.

```
order_id,sku,quantity,unit_price_cents
A,X,1,10
B,Y,2,20
A,X,3,30
```

```
[
  {"order_id":"A","lines":[
    {"sku":"X","quantity":1,"unit_price_cents":10},
    {"sku":"X","quantity":3,"unit_price_cents":30}
  ]},
  {"order_id":"B","lines":[
    {"sku":"Y","quantity":2,"unit_price_cents":20}
  ]}
]
```

T3 - refactor có ràng buộc hành vi. Code gốc duyệt input nhiều lần và lặp lại logic cộng tiền theo khách hàng. Ngoài generator thông thường, test dùng một iterable ném lỗi khi `__iter__` bị gọi lần thứ hai. Các trường hợp khác kiểm tra paid order có tổng 0, đơn pending/cancelled, thứ tự customer key và việc giữ nguyên dữ liệu đầu vào. Cách dùng helper được xem thêm trong mã nguồn sau chỉnh sửa. Code gốc · Prompt T3 · Test T3

3.2. Cấu hình và quy trình chạy

Thành phần	Cấu hình dùng trong lượt đo
OpenCode	`2.2.7`, chạy một server lab riêng và điều khiển qua Promise SDK
Pi	`1.5.0`, CLI JSON mode, chạy bằng Bun `1.4.4`
Codex CLI	`0.166.0-alpha.6`, binary có sẵn trên máy, `exec --json --ephemeral`
Model	`gpt-6.1-sol` cho cả ba tool
Reasoning	`medium`
Ngôn ngữ và dependency của task	Python 3.12, standard library
Giới hạn thời gian	240 giây cho mỗi lượt agent
Số lần lặp	3 lượt cho mỗi tổ hợp tool-task
Thực thi	Tuần tự, xoay thứ tự tool theo task và lần lặp
Workspace	Bản sao sạch dưới `/tmp/coding-benchmark/workspaces/<run_id>`

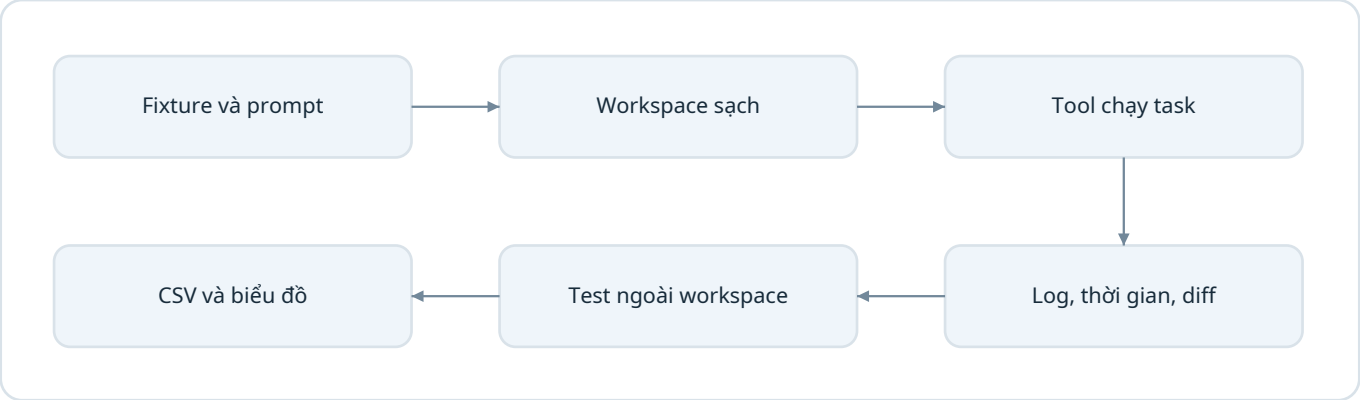
Cả ba tool đã vượt qua smoke test bằng model được chọn trước khi bắt đầu task. Ban đầu đã thử dùng model miễn phí của OpenCode qua endpoint trực tiếp; endpoint đó trả HTTP 429. Bộ số liệu chính dùng đăng nhập OpenAI hiện có của từng công cụ. Pi nhập credential từ đăng nhập Codex vào thư mục riêng. OpenCode nhập đúng credential OpenAI đang hoạt động vào DB lab qua API. Credential nằm trong `/tmp`, còn thư mục kết quả chứa dữ liệu đo và mã nguồn tái lập. Chuẩn bị auth riêng · Server lab · Smoke Codex · Smoke Pi · Smoke OpenCode

Mỗi lượt nhận cùng prompt task, cùng phần nhắc đọc README và cùng nội dung repo. Pi dùng `--no-extensions --no-skills --no-session`. Codex dùng thư mục cấu hình benchmark riêng, `workspace-write` và chế độ không hỏi phê duyệt. OpenCode cho phép các thao tác của task trong cấu hình lab. Các system prompt và định nghĩa tool gốc vẫn thuộc về từng harness. Đây là những khác biệt của sản phẩm cần quan sát trong thực nghiệm. Câu lệnh và cấu hình trong runner · Adapter OpenCode

Một lượt chạy có trình tự sau.

1. Sao chép fixture và tạo commit baseline với timestamp cố định.
2. Mở session mới, gửi đúng prompt và bắt đầu đồng hồ đo.
3. Thu stdout/event JSONL và stderr đến khi công cụ dừng hoặc timeout.
4. Chạy test nghiệm thu từ bên ngoài workspace, rồi chạy lại toàn bộ thư mục test của repo.
5. Kiểm tra file test gốc vẫn giữ nguyên, lưu diff và snapshot bản sửa.
6. Tính token từ event gốc và ghi `metrics.json` cho lượt đó.

Một lượt được tính là đạt bộ nghiệm thu chính khi công cụ kết thúc thành công trong thời hạn, toàn bộ test nghiệm thu và test của repository đều đạt, đồng thời file test gốc được giữ nguyên. Kết quả chấm dựa trên dữ liệu thực thi và bản sửa đã lưu. Grader · Điều kiện terminal và nghiệm thu



Hình 5. Quy trình của benchmark. Test nghiệm thu là code do harness thực thi sau khi agent kết thúc. Runner · Phân tích số liệu

3.3. Cách đo token và thời gian

Thời gian là wall-clock từ lúc tạo tiến trình chạy task đến khi tiến trình đó kết thúc. Nó bao gồm đọc repository, gọi model, thực thi tool, các test do agent tự chạy và ghi đầu ra. Việc chuẩn bị workspace và chạy grader sau đó nằm ngoài đồng hồ. OpenCode dùng server đã khởi động, còn Pi và Codex đi qua CLI mới ở mỗi lượt. Vì thế, số đo phản ánh các entry point được chọn trong lab. Hàm launch

Token được chuẩn hóa trước khi so sánh. OpenCode ghi `input` không cache và tách output nhìn thấy khỏi reasoning. Pi cũng tách input không cache, nhưng `output` của nó đã bao gồm reasoning. Codex trả tổng input và cached input theo trường riêng trong `turn.completed`; reasoning là một phần của output. Cộng cached input vào `input_tokens` của Codex hoặc cộng reasoning thêm vào output của Pi sẽ làm số liệu bị tính hai lần. OpenCode usage · Pi usage · Codex JSONL usage

```
OpenCode input_total  = input + cache.read + cache.write
OpenCode output_total = output + reasoning

Pi input_total        = input + cacheRead + cacheWrite
Pi output_total       = output

Codex input_total     = input_tokens
Codex output_total    = output_tokens

total_tokens          = input_total + output_total
```

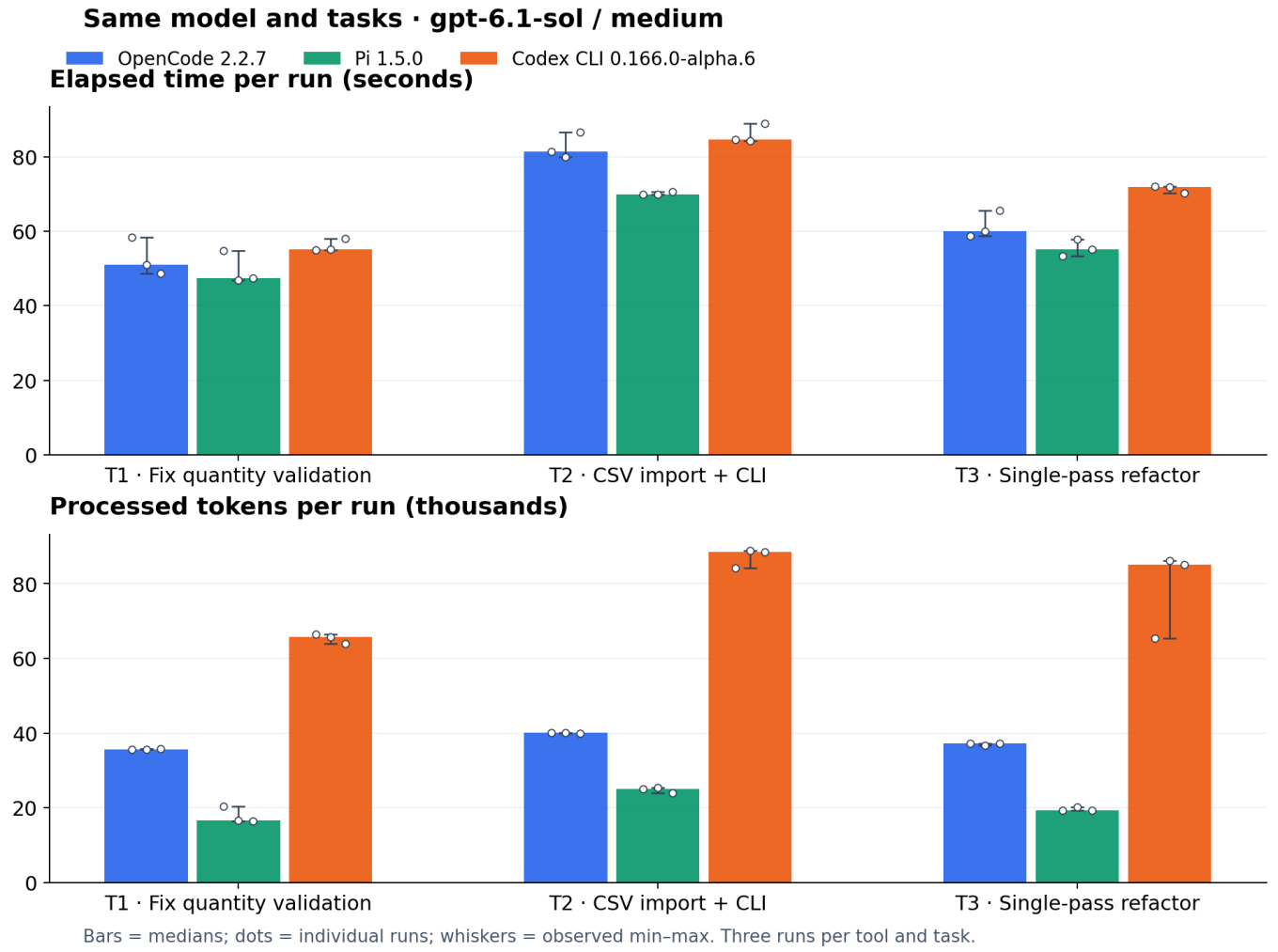
OpenCode được cộng từ từng `session.step.ended`; Pi được cộng từ mỗi `message_end` của assistant. Những event delta và bản sao message trong `agent_end` được bỏ qua khi cộng usage. Codex dùng usage tổng hợp của `turn.completed`. Dữ liệu giữ thêm input không cache, input cache và reasoning để người đọc kiểm tra cách chuẩn hóa. Mã tính usage · CSV chuẩn hóa

Token trong bảng là tổng token được xử lý qua các lượt model của task, bao gồm phần cache. Nó mô tả kích thước ngữ cảnh được dùng trong toàn bộ lượt chạy. Bảng thành phần token bên dưới tách phần cache để đánh giá rõ hơn lượng input mới. Các số này được lấy từ telemetry; báo cáo không quy đổi chúng thành chi phí tiền tệ.

3.4. Kết quả tổng hợp

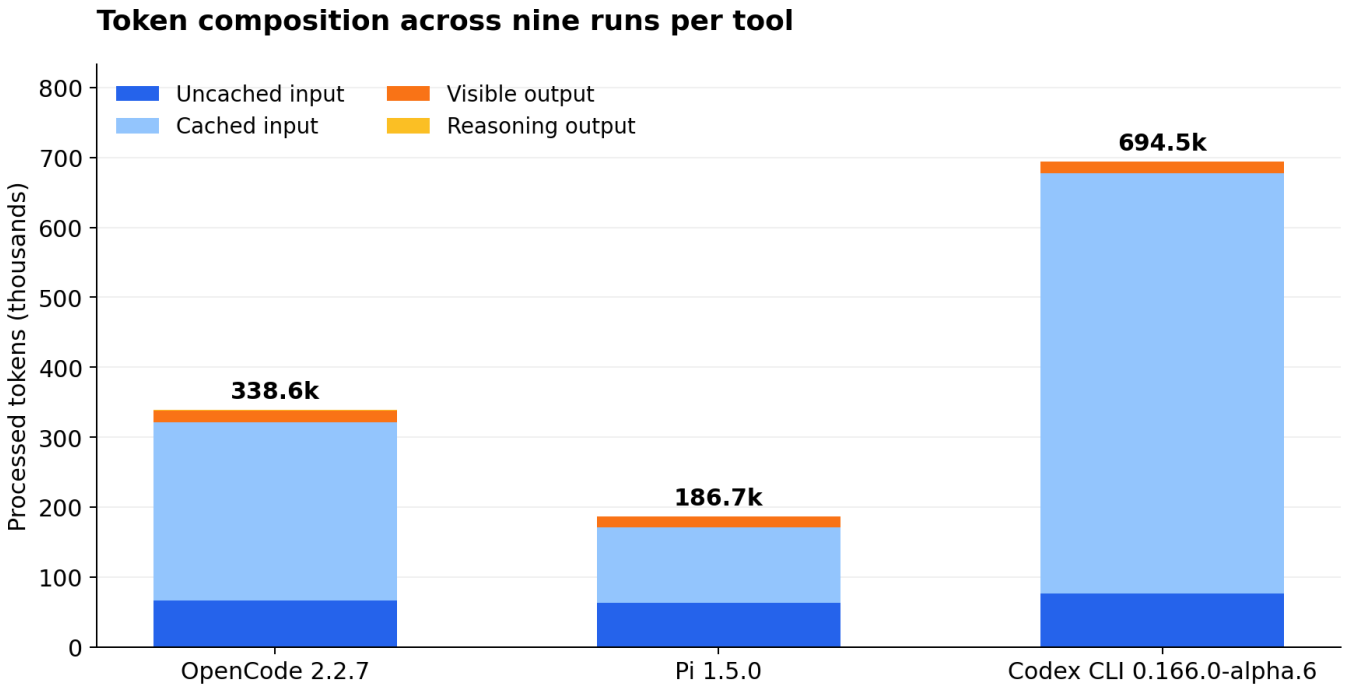
Tool	Lượt đạt	Tỉ lệ	Trung vị thời gian	Trung vị tổng token
OpenCode 2.2.7	9/9	100%	60,0 s	37.190
Pi 1.5.0	9/9	100%	55,2 s	20.255
Codex CLI 0.166.0-alpha.6	9/9	100%	71,8 s	84.184

Task	Tool	Lượt đạt	Thời gian trung vị [min-max], giây	Token trung vị [min-max]
T1	OpenCode 2.2.7	3/3	51,0 [48,6-58,3]	35.640 [35.615-35.845]
T1	Pi 1.5.0	3/3	47,3 [46,9-54,9]	16.585 [16.438-20.429]
T1	Codex CLI 0.166.0-alpha.6	3/3	55,2 [55,0-58,0]	65.801 [64.008-66.470]
T2	OpenCode 2.2.7	3/3	81,3 [79,9-86,6]	40.045 [40.008-40.194]
T2	Pi 1.5.0	3/3	69,9 [69,9-70,6]	25.140 [23.965-25.350]
T2	Codex CLI 0.166.0-alpha.6	3/3	84,5 [84,2-89,0]	88.516 [84.184-88.789]
T3	OpenCode 2.2.7	3/3	60,0 [58,7-65,5]	37.190 [36.773-37.259]
T3	Pi 1.5.0	3/3	55,2 [53,3-57,8]	19.316 [19.245-20.255]
T3	Codex CLI 0.166.0-alpha.6	3/3	71,8 [70,1-72,1]	85.160 [65.387-86.149]



Hình 6. Cột thể hiện trung vị, chấm thể hiện từng lượt chạy và thanh khoảng thể hiện min-max quan sát được. Mỗi cột có ba mẫu. SVG · Số liệu nguồn

Tool	Tổng input mới	Tổng input cache	Tổng output, gồm reasoning	Tổng token của 9 lượt
OpenCode 2.2.7	65.993	255.616	16.960	338.569
Pi 1.5.0	63.088	107.520	16.115	186.723
Codex CLI 0.166.0-alpha.6	76.180	600.832	17.452	694.464



Hình 7. Tổng token của chín lượt trên mỗi tool, tách input mới, input cache, output nhìn thấy và reasoning. SVG · Thống kê nguồn

3.5. Dữ liệu từng lượt và đánh giá kết quả

Lượt	Test chính	Test repo	Kết quả	Giây	Input, gồm cache	Output, gồm reasoning	Tổng token
opencode-T1-r1	9/9	Đạt	Đạt	58,3	34.409	1.206	35.615
pi-T1-r1	9/9	Đạt	Đạt	54,9	18.998	1.431	20.429
codex-T1-r1	9/9	Đạt	Đạt	55,0	65.195	1.275	66.470
opencode-T1-r2	9/9	Đạt	Đạt	51,0	34.429	1.211	35.640
pi-T1-r2	9/9	Đạt	Đạt	46,9	15.416	1.169	16.585
codex-T1-r2	9/9	Đạt	Đạt	55,2	64.639	1.162	65.801
opencode-T1-r3	9/9	Đạt	Đạt	48,6	34.577	1.268	35.845
pi-T1-r3	9/9	Đạt	Đạt	47,3	15.304	1.134	16.438
codex-T1-r3	9/9	Đạt	Đạt	58,0	62.646	1.362	64.008

Lượt	Test chính	Test repo	Kết quả	Giây	Input, gồm cache	Output, gồm reasoning	Tổng token
opencode-T2-r1	12/12	Đạt	Đạt	81,3	37.573	2.621	40.194
pi-T2-r1	12/12	Đạt	Đạt	69,9	22.774	2.366	25.140
codex-T2-r1	12/12	Đạt	Đạt	84,5	81.524	2.660	84.184
opencode-T2-r2	12/12	Đạt	Đạt	79,9	37.419	2.626	40.045
pi-T2-r2	12/12	Đạt	Đạt	69,9	22.924	2.426	25.350
codex-T2-r2	12/12	Đạt	Đạt	84,2	86.304	2.485	88.789
opencode-T2-r3	12/12	Đạt	Đạt	86,6	37.378	2.630	40.008
pi-T2-r3	12/12	Đạt	Đạt	70,6	21.602	2.363	23.965
codex-T2-r3	12/12	Đạt	Đạt	89,0	85.816	2.700	88.516

Lượt	Test chính	Test repo	Kết quả	Giây	Input, gồm cache	Output, gồm reasoning	Tổng token
opencode-T3-r1	8/8	Đạt	Đạt	58,7	35.432	1.827	37.259
pi-T3-r1	8/8	Đạt	Đạt	53,3	17.510	1.735	19.245
codex-T3-r1	8/8	Đạt	Đạt	72,1	63.411	1.976	65.387
opencode-T3-r2	8/8	Đạt	Đạt	60,0	35.066	1.707	36.773
pi-T3-r2	8/8	Đạt	Đạt	57,8	18.512	1.743	20.255
codex-T3-r2	8/8	Đạt	Đạt	71,8	84.271	1.878	86.149
opencode-T3-r3	8/8	Đạt	Đạt	65,5	35.326	1.864	37.190
pi-T3-r3	8/8	Đạt	Đạt	55,2	17.568	1.748	19.316
codex-T3-r3	8/8	Đạt	Đạt	70,1	83.206	1.954	85.160

Mức độ hoàn thành. Có 27/27 lượt đạt tiêu chí nghiệm thu, tương ứng 261/261 test nghiệm thu đạt. Mỗi công cụ có chín lượt, phân bố đều trên ba loại task. Cả ba công cụ đều đạt bộ test chính ở cả chín lượt. Kết quả này cần được đọc cùng kiểm tra bổ sung ở cuối mục để đánh giá độ đầy đủ của bản sửa. Số liệu tổng hợp · Chấm lại snapshot

Thời gian. Trung vị toàn bộ chín lượt thấp nhất thuộc Pi, ở mức 55,2 giây. Pi cũng có trung vị thời gian thấp nhất ở từng task. Bảng min-max và từng chấm trên biểu đồ cho biết độ dao động thực tế. Số đo bao gồm cả lựa chọn entry point và runtime của lab. Dữ liệu từng lượt

Tổng token. Để thực hiện cùng chín lượt task, Pi dùng tổng cộng 186.723 token đã chuẩn hóa, thấp hơn 44,8% so với OpenCode, 73,1% so với Codex CLI. Các giá trị này bao gồm input cache. Khi chỉ xét input mới, OpenCode dùng 65.993 token, Pi dùng 63.088 và Codex dùng 76.180. Theo thước đo này, Pi thấp hơn khoảng 4,4% so với OpenCode và 17,2% so với Codex; mức chênh lệch nhỏ hơn đáng kể so với tổng token bao gồm cache. Thành phần usage

Một quan sát từ smoke test giúp giải thích một phần chênh lệch. Với cùng yêu cầu chỉ trả `BENCHMARK_READY`, input được báo về là 4.470 token ở OpenCode, 1.062 ở Pi và 14.333 ở Codex. Các harness đưa vào ngữ cảnh khởi đầu khác nhau ngay trước khi agent cần đọc code. Trong task nhiều lượt, phần prefix và lịch sử tiếp tục xuất hiện trong các request sau. Việc tách riêng ảnh hưởng của system prompt, tool schema và chiến lược lấy code cần một phép đo bổ sung. Smoke OpenCode · Smoke Pi · Smoke Codex

Bản sửa và chất lượng triển khai. Ở các snapshot lượt đầu, T1 được sửa bằng cách phân biệt giá trị thiếu với quantity được truyền vào, đồng thời giữ phần kiểm tra kiểu hiện có. T2 dùng parser `csv` của standard library, kiểm tra tập header và giới hạn chữ số bằng ký tự ASCII. Ở T3, kiểm tra source tìm thấy helper được cả hai public function gọi trong 9/9 snapshot. Các bản sửa lượt đầu dùng một vòng lặp cộng dồn, nên dữ liệu generator được xử lý trực tiếp. OpenCode T1 diff · Pi T2 source · Codex T3 source · Kiểm tra helper

Kiểm tra bổ sung sau khi đọc code. Cùng một probe thêm dòng trống trước header CSV được chạy trên toàn bộ chín snapshot T2. Đây là đầu vào bổ sung ngoài 12 test T2 đã cố định. Kết quả dưới đây ghi hành vi của từng bản sửa và được giữ riêng với tỉ lệ hoàn thành của bộ test chính.

Tool	Chấp nhận dòng trống trước header và trả đúng output	Hai lệnh CLI cũ vẫn trả đúng output
OpenCode	0/3	3/3
Pi	1/3	3/3
Codex CLI	2/3	3/3

Yêu cầu bỏ qua dòng trống chưa được suite chính kiểm tra ở vị trí trước header. Những snapshot từ chối đầu vào này chưa đáp ứng cách hiểu bỏ qua dòng trống ở mọi vị trí. Điểm 100% của suite chính cần được đọc cùng kết quả probe bổ sung này. Đầu vào bổ sung được áp dụng đồng đều cho cả chín snapshot T2 sau khi đọc code; các lượt gọi model và bản sửa gốc được giữ nguyên. Code probe · Kết quả đầy đủ

Trong bộ nhiệm vụ này, Pi dùng ít tổng token và có thời gian trung vị thấp nhất. Kết quả kiểm tra bổ sung cho thấy độ đầy đủ của parser vẫn khác nhau giữa các bản sửa, kể cả khi điểm của bộ test chính bằng nhau. Vì vậy, đánh giá một công cụ cần kết hợp số liệu tài nguyên với test và việc đọc lại mã nguồn. Khi mở rộng sang dự án lớn, cần xây dựng bộ nhiệm vụ phù hợp và tiếp tục lưu test, log cùng diff.

3.6. Phạm vi diễn giải kết quả

Bộ thử gồm ba task nhỏ và ba lần lặp cho mỗi tổ hợp tool-task. Task dùng Python standard library, ít file và không cần dịch vụ ngoài. Kết quả phù hợp để quan sát việc tìm file, sửa code, chạy test và xử lý các trường hợp biên đã nêu. Khả năng làm việc trên codebase lớn, dependency phức tạp, giao diện người dùng và tác vụ kéo dài cần bộ dữ liệu khác để đánh giá.

Cùng model và mức reasoning giúp giảm một nguồn chênh lệch, trong khi system prompt, tool schema, cách chọn context và protocol gọi model vẫn khác theo harness. OpenCode và Pi/Codex dùng các đường tích hợp native của chúng. Cache phía provider và trạng thái mạng cũng thay đổi giữa các lượt. Thứ tự xoay vòng làm các tool có cơ hội xuất hiện ở từng vị trí, còn ba lần lặp mới chỉ cho phép mô tả độ dao động quan sát được. Lịch chạy · Manifest

Codex CLI được thử nghiệm bằng bản alpha có sẵn trong ứng dụng. Pi chạy bằng Bun, còn server OpenCode đã được khởi động trước mỗi đợt đo. Những lựa chọn này được giữ cố định trong bộ số liệu và cần được giữ lại khi tái lập. Nhận xét về thời gian áp dụng cho cấu hình thực nghiệm này.

3.7. Mã chạy đầy đủ và cách tái lập

Đường dẫn	Vai trò
`fixture/`	Repository đầu vào và bốn test gốc
`prompts/T1.txt` đến `T3.txt`	Yêu cầu nguyên văn
`acceptance/T1.py` đến `T3.py`	Toàn bộ test nghiệm thu
`dataset.json`	Danh sách test, baseline và hash fixture
`scripts/prepare_auth.py`	Chuẩn bị đăng nhập riêng trong `/tmp`
`scripts/server.py`	Khởi động và dừng server OpenCode riêng
`scripts/run-opencode.ts`	Gửi task và thu event qua SDK
`scripts/run_benchmark.py`	Chạy tool, đo thời gian, chấm test và lưu bản sửa
`scripts/grade.py`	Chạy test nghiệm thu và kiểm tra file test gốc
`scripts/analyze.py`	Chuẩn hóa usage và xuất thống kê, CSV
`scripts/plot_results.py`	Xuất biểu đồ PNG/SVG
`results/<run_id>/`	Log, metrics, grade, diff và solution của từng lượt

Chạy từ `experiments/tool-benchmark`, trên Linux x64 với Python 3, Git, binary Codex phù hợp và đăng nhập OpenAI của các tool. Binary OpenCode và Bun có thể cài lại bằng script setup của phần trước.

```
python3 ../opencode-v1-v2/scripts/setup.py
/tmp/opencode-lab/runtime/bun-linux-x64/bun install --frozen-lockfile
python3 scripts/prepare_auth.py
python3 scripts/run_benchmark.py --rounds 3
python3 scripts/analyze.py
python3 scripts/verify_results.py
python3 scripts/audit_saved_solutions.py
python3 scripts/plot_results.py
```

Biểu đồ cần dependency trong `requirements-plot.txt`. Có thể cài bằng pip vào môi trường Python dùng để vẽ. Runner nhận `BENCH_CODEX_BIN` nếu binary Codex ở vị trí khác. `BENCH_CODEX_AUTH` và `BENCH_OPENCODE_DB` cho phép chỉ định vị trí đăng nhập cục bộ khi chúng khác mặc định. Package và lockfile · Dependency biểu đồ

Runner bỏ qua lượt đã có `metrics.json`. Khi chạy lại trên máy khác, dùng một bản sao thư mục benchmark có `results/` rỗng và thư mục workspace mới. Toàn bộ code nghiệm thu nằm trong gói báo cáo để có thể chấm lại `solution/` đã lưu mà không cần gọi model thêm.